

SEGMENT TREE

1. Overview

A **Segment Tree** is a binary tree data structure used for storing information about intervals or segments. It allows querying aggregate information (like sum, min, max, gcd) over a range in **$O(\log n)$** time and updating a single element or a range in **$O(\log n)$** time.

Think of it as a decision tree over an array. Each node represents a segment of the array, and the root represents the entire array. The leaves represent individual elements. Internal nodes store the combined result (merge) of their children — sum, min, max, or any custom operation.

Segment Trees are the go-to data structure when you need to answer **range queries with point or range updates** and the operation is **associative** (order of grouping doesn't matter).

2. Intuition

Simple Explanation

You have an array of size n . You want to answer queries like "what is the sum of elements from index 2 to 7?" or "what is the minimum value in range [3, 10]?" If you answer naively, each query takes $O(n)$. If you precompute prefix sums, updates take $O(n)$. Segment tree gives you both query and update in $O(\log n)$.

Analogy

Imagine a **tournament bracket** for a sports league. Each match combines two team scores into a result. The final match (root) gives the overall winner/aggregate. If one team's score changes (point update), you only recompute the matches on the path from that leaf to the root — $O(\log n)$ matches. If you want to know the result of a range of teams, you combine the results of $O(\log n)$ disjoint segments that exactly cover your range.

Step-by-Step Reasoning

1. Take an array of size n .
2. Build a binary tree where each node represents a contiguous segment $[l, r]$.
3. The root represents $[0, n-1]$.
4. A node's left child represents $[l, mid]$, right child represents $[mid+1, r]$.
5. Each leaf represents a single element $[i, i]$.
6. Each internal node stores the combined result of its two children.
7. To query a range, traverse the tree: if a node's segment is fully inside the query range, return its value; if partially overlapping, recurse into children.
8. To update, go down to the leaf, update it, then recompute all ancestors.

Why It Works

- The tree has height $\text{ceil}(\log_2(n))$.
- Any contiguous range can be expressed as a union of **$O(\log n)$** disjoint node segments.

- This is the key insight: the decomposition of any range into tree nodes is logarithmic.

3. When to Use It

Use a Segment Tree when you need:

Situation	Example
Range query with point updates	Sum/min/max over $[1, r]$, update one element
Range query with range updates	Add a value to all elements in $[1, r]$, then query sum
Custom associative merge operation	XOR, gcd, product, or any custom combine
Dynamic allocation for sparse arrays	Array size up to 10^9 but only few updates
Persistent data structures	Query historical versions of the array
2D range queries	Queries on submatrices

Common Trigger Phrases

- "Range sum query"
- "Range minimum query"
- "Range update, point query"
- "Range update, range query"
- "Lazy propagation"
- "Queries on subarray"
- "Point update, range query"
- "You are given an array and Q queries..."

4. When Not to Use It

Situation	Better Alternative	Reason
Only prefix queries (no updates)	Prefix sum array	$O(1)$ query, $O(n)$ build
Only point updates and prefix queries	Fenwick Tree (BIT)	Simpler, less memory, faster constant
Only range min, static array	Sparse Table	$O(1)$ query, $O(n \log n)$ build
Need order statistics on dynamic set	Order Statistic Tree / PBDS	Built for that purpose
Massive memory constraints	Fenwick Tree or Sparse Table	Segment tree uses $4n$ memory
Only need range sum with no updates	Prefix sum array	Simpler, $O(1)$ query
Queries are only on small ranges	Brute force	Lower constant, simpler code

Situation	Better Alternative	Reason
Random updates and queries are rare	SQRT Decomposition	$O(\sqrt{n})$ per op, simpler

Common Wrong Assumptions

- **"Segment Tree is always faster."** — No, for simple problems Fenwick Tree is faster and uses less memory.
- **"Segment Tree can handle any operation."** — Only **associative** operations. Subtraction is fine, but not non-associative ops.
- **"Segment Tree is the best for range min."** — For static arrays, Sparse Table gives $O(1)$ query.

5. Core Concepts

5.1 Node Representation

Each node stores:

- `l, r` — segment boundaries (implicit or explicit)
- `val` — aggregate value for this segment (sum, min, max, etc.)
- `lazy` — pending update value (for lazy propagation)

Why it matters: This is the building block. Everything else is built on how nodes are structured.

5.2 Merge Function

A function that combines two child values into a parent value.

```
int merge(int a, int b) { return a + b; } // sum
int merge(int a, int b) { return min(a, b); } // min
int merge(int a, int b) { return max(a, b); } // max
```

Why it matters: The merge function defines the query type. By parameterizing it, one segment tree can handle any associative operation.

5.3 Tree Array Representation

Segment trees are stored in a flat array of size $4 * n$ (safe upper bound).

- Root at index `1`
- Left child of `i`: `2*i`
- Right child of `i`: `2*i+1`

Why it matters: This is the standard array-based binary tree representation. It avoids explicit pointers and is cache-friendly.

5.4 Range Decomposition

When querying $[ql, qr]$, we decompose it into $O(\log n)$ disjoint node segments.

Three cases at each node:

- **No overlap:** $[l, r]$ is completely outside $[ql, qr]$ → return identity
- **Full overlap:** $[l, r]$ is completely inside $[ql, qr]$ → return node value
- **Partial overlap:** Split and recurse to children

Why it matters: Understanding this decomposition is the key to understanding how segment tree queries work.

5.5 Lazy Propagation

A technique to defer range updates. Instead of updating all leaves in a range, we mark a node as "lazy" and propagate only when needed.

Why it matters: Without lazy propagation, range updates would be $O(n \log n)$. With it, they are $O(\log n)$ — the same as a query.

5.6 Identity Element

The element that doesn't affect the operation:

- Sum: 0
- Min: INT_MAX / ∞
- Max: $INT_MIN / -\infty$
- XOR: 0
- Product: 1

Why it matters: When a node has no overlap with the query range, we return the identity element. It must be correctly chosen.

5.7 Persistent Segment Tree

A segment tree that preserves all historical versions after each update. Instead of modifying in-place, we create new nodes along the update path.

Why it matters: Allows querying any previous version of the data structure. Essential for problems like "K-th smallest in range" or "range queries on a dynamic array over time".

5.8 Dynamic Segment Tree

A segment tree where nodes are created on-demand, not pre-allocated. Used when the array size is too large (e.g., 10^9) but the number of updates/queries is manageable.

Why it matters: Saves memory for sparse arrays. Instead of $4 * n$ memory, we use $O(q \log n)$ where q is the number of operations.

6. Step-by-Step Algorithm

6.1 Build

1. Start with array `arr` of size `n`.
2. Allocate array `tree` of size `4 * n`.
3. Call `build(1, 0, n-1)`:
 - If `l == r` (leaf): `tree[node] = arr[l]`
 - Else:
 - `mid = (l + r) / 2`
 - Build left child: `build(2*node, l, mid)`
 - Build right child: `build(2*node+1, mid+1, r)`
 - `tree[node] = merge(tree[2*node], tree[2*node+1])`

6.2 Point Update

1. Call `update(1, 0, n-1, pos, val)`:
 - If `l == r` (leaf): `tree[node] = val` (or `tree[node] += val` for increment)
 - Else:
 - `mid = (l + r) / 2`
 - If `pos <= mid`: update left child
 - Else: update right child
 - `tree[node] = merge(tree[2*node], tree[2*node+1])`

6.3 Range Query

1. Call `query(1, 0, n-1, ql, qr)`:
 - If `ql > r || qr < l`: return identity (no overlap)
 - If `ql <= l && r <= qr`: return `tree[node]` (full overlap)
 - Else:
 - `mid = (l + r) / 2`
 - `left = query(2*node, l, mid, ql, qr)`
 - `right = query(2*node+1, mid+1, r, ql, qr)`
 - Return `merge(left, right)`

6.4 Range Update with Lazy Propagation

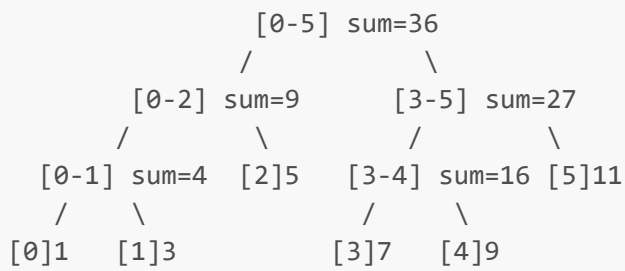
1. When updating range `[ul, ur]` with value `val`:
 - If no overlap: return
 - If full overlap: set `tree[node] += val * (r-l+1)` (for sum), mark `lazy[node] += val`
 - Else: push lazy to children, recurse, then recompute node value
2. When querying with lazy:
 - Before recursing, push lazy to children
 - Then proceed as normal range query

7. Dry Run

Problem: Range Sum Query with Point Updates

Array: `[1, 3, 5, 7, 9, 11]` (`n = 6`)

Building the Tree



Step-by-step build:

Node	Segment	Children	Computation	Value
6	[5,5]	leaf	arr[5]	11
5	[4,4]	leaf	arr[4]	9
4	[3,3]	leaf	arr[3]	7
3	[2,2]	leaf	arr[2]	5
2	[1,1]	leaf	arr[1]	3
1	[0,0]	leaf	arr[0]	1
7	[3,4]	4,5	7+9	16
8	[0,1]	1,2	1+3	4
9	[3,5]	7,6	16+11	27
10	[0,2]	8,3	4+5	9
11	[0,5]	10,9	9+27	36

Query: sum in range [1, 4]

Node	Segment	Case	Action	Result
11	[0,5]	Partial	Recurse	-
10	[0,2]	Partial	Recurse	-
8	[0,1]	Partial	Recurse	-
1	[0,0]	No overlap	Return 0	0
2	[1,1]	Full (1 ∈ [1,4])	Return 3	3
8	[0,1]	merge	0+3	3
3	[2,2]	Full (2 ∈ [1,4])	Return 5	5
10	[0,2]	merge	3+5	8

Node	Segment	Case	Action	Result
9	[3,5]	Partial	Recurse	-
7	[3,4]	Full (3,4 ∈ [1,4])	Return 16	16
6	[5,5]	No overlap	Return 0	0
9	[3,5]	merge	16+0	16
11	[0,5]	merge	8+16	24

Answer: $1 + 3 + 5 + 7 + 9 = 24$ ✓

Update: set position 2 to 10

Step	Node	Segment	Action	New Value
1	11	[0,5]	$2 \leq \text{mid}$? mid=2, yes → recurse left	-
2	10	[0,2]	$2 \leq \text{mid}$? mid=1, no → recurse right	-
3	3	[2,2]	leaf, set to 10	10
4	10	[0,2]	merge(8=4, 3=10)	14
5	11	[0,5]	merge(10=14, 9=27)	41

New tree root sum: 41 ✓ (old sum 36 - 5 + 10 = 41)

8. C++ Implementation

Basic Segment Tree — Range Sum Query, Point Update

```
#include <bits/stdc++.h>
using namespace std;

class SegmentTree {
private:
    int n;
    vector<int> tree;

    void build(const vector<int>& arr, int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(arr, 2 * node, l, mid);
        build(arr, 2 * node + 1, mid + 1, r);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}
```

```

void update(int node, int l, int r, int pos, int val) {
    if (l == r) {
        tree[node] = val;
        return;
    }
    int mid = (l + r) / 2;
    if (pos <= mid)
        update(2 * node, l, mid, pos, val);
    else
        update(2 * node + 1, mid + 1, r, pos, val);
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

int query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0; // no overlap
    if (ql <= l && r <= qr) return tree[node]; // full overlap
    int mid = (l + r) / 2;
    int left = query(2 * node, l, mid, ql, qr);
    int right = query(2 * node + 1, mid + 1, r, ql, qr);
    return left + right;
}

public:
    SegmentTree(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }

    void pointUpdate(int pos, int val) {
        update(1, 0, n - 1, pos, val);
    }

    int rangeQuery(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
};

int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11};
    SegmentTree st(arr);

    cout << "Sum [1,4]: " << st.rangeQuery(1, 4) << "\n"; // 24
    st.pointUpdate(2, 10);
    cout << "After update, sum [1,4]: " << st.rangeQuery(1, 4) << "\n"; // 29
    return 0;
}

```

Range Minimum Query — Point Update

```

#include <bits/stdc++.h>
using namespace std;

class SegTreeRMQ {
private:
    int n;
    vector<int> tree;
    const int INF = 1e9;

    void build(const vector<int>& arr, int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(arr, 2 * node, l, mid);
        build(arr, 2 * node + 1, mid + 1, r);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }

    void update(int node, int l, int r, int pos, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid)
            update(2 * node, l, mid, pos, val);
        else
            update(2 * node + 1, mid + 1, r, pos, val);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }

    int query(int node, int l, int r, int ql, int qr) {
        if (ql > r || qr < l) return INF;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return min(query(2 * node, l, mid, ql, qr),
                    query(2 * node + 1, mid + 1, r, ql, qr));
    }

public:
    SegTreeRMQ(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }

    void pointUpdate(int pos, int val) {
        update(1, 0, n - 1, pos, val);
    }
}

```

```

int rangeMin(int l, int r) {
    return query(1, 0, n - 1, l, r);
}

};

int main() {
    vector<int> arr = {5, 2, 8, 1, 9, 3};
    SegTreeRMQ st(arr);
    cout << "Min [1,4]: " << st.rangeMin(1, 4) << "\n"; // 1
    st.pointUpdate(3, 10);
    cout << "After update, min [1,4]: " << st.rangeMin(1, 4) << "\n"; // 2
    return 0;
}

```

Segment Tree with Lazy Propagation — Range Update, Range Sum

```

#include <bits/stdc++.h>
using namespace std;

class LazySegmentTree {
private:
    int n;
    vector<long long> tree, lazy;

    void build(const vector<int>& arr, int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(arr, 2 * node, l, mid);
        build(arr, 2 * node + 1, mid + 1, r);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    void apply(int node, int l, int r, long long val) {
        tree[node] += val * (r - l + 1);
        lazy[node] += val;
    }

    void push(int node, int l, int r) {
        if (lazy[node] != 0) {
            int mid = (l + r) / 2;
            apply(2 * node, l, mid, lazy[node]);
            apply(2 * node + 1, mid + 1, r, lazy[node]);
            lazy[node] = 0;
        }
    }

    void rangeUpdate(int node, int l, int r, int ul, int ur, int val) {
        if (ul > r || ur < l) return;
    }
}

```

```

        if (ul <= l && r <= ur) {
            apply(node, l, r, val);
            return;
        }
        push(node, l, r);
        int mid = (l + r) / 2;
        rangeUpdate(2 * node, l, mid, ul, ur, val);
        rangeUpdate(2 * node + 1, mid + 1, r, ul, ur, val);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    long long rangeQuery(int node, int l, int r, int ql, int qr) {
        if (ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return tree[node];
        push(node, l, r);
        int mid = (l + r) / 2;
        return rangeQuery(2 * node, l, mid, ql, qr) +
            rangeQuery(2 * node + 1, mid + 1, r, ql, qr);
    }

public:
    LazySegmentTree(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n, 0);
        lazy.resize(4 * n, 0);
        build(arr, 1, 0, n - 1);
    }

    void rangeAdd(int l, int r, int val) {
        rangeUpdate(1, 0, n - 1, l, r, val);
    }

    long long rangeSum(int l, int r) {
        return rangeQuery(1, 0, n - 1, l, r);
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    LazySegmentTree st(arr);

    cout << "Sum [0,4]: " << st.rangeSum(0, 4) << "\n"; // 15
    st.rangeAdd(1, 3, 10); // add 10 to indices 1,2,3
    cout << "Sum [0,4]: " << st.rangeSum(0, 4) << "\n"; // 15 + 30 = 45
    cout << "Sum [2,4]: " << st.rangeSum(2, 4) << "\n"; // (3+10)+(4)+(5) = 22
    return 0;
}

```

Segment Tree with Custom Merge Function

```

#include <bits/stdc++.h>
using namespace std;

class SegmentTreeCustom {
private:
    int n;
    vector<int> tree;
    function<int(int, int)> merge;

    void build(const vector<int>& arr, int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(arr, 2 * node, l, mid);
        build(arr, 2 * node + 1, mid + 1, r);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }

    void update(int node, int l, int r, int pos, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid)
            update(2 * node, l, mid, pos, val);
        else
            update(2 * node + 1, mid + 1, r, pos, val);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }

    int query(int node, int l, int r, int ql, int qr, int identity) {
        if (ql > r || qr < l) return identity;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return merge(query(2 * node, l, mid, ql, qr, identity),
                     query(2 * node + 1, mid + 1, r, ql, qr, identity));
    }

public:
    SegmentTreeCustom(const vector<int>& arr, function<int(int, int)> mergeFn) {
        n = arr.size();
        merge = mergeFn;
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }

    void pointUpdate(int pos, int val) {
        update(1, 0, n - 1, pos, val);
    }
}

```

```

    int rangeQuery(int l, int r, int identity) {
        return query(1, 0, n - 1, l, r, identity);
    }
};

int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11};

    // GCD segment tree
    SegmentTreeCustom gcdTree(arr, [](int a, int b) { return __gcd(a, b); });
    cout << "GCD [1,4]: " << gcdTree.rangeQuery(1, 4, 0) << "\n"; //
gcd(3,5,7,9)=1

    // XOR segment tree
    SegmentTreeCustom xorTree(arr, [](int a, int b) { return a ^ b; });
    cout << "XOR [0,3]: " << xorTree.rangeQuery(0, 3, 0) << "\n"; // 1^3^5^7 = 0

    // Max segment tree
    SegmentTreeCustom maxTree(arr, [](int a, int b) { return max(a, b); });
    cout << "Max [1,4]: " << maxTree.rangeQuery(1, 4, INT_MIN) << "\n"; // 9
    return 0;
}

```

Segment Tree Beats (Range Chmin/Chmax)

```

#include <bits/stdc++.h>
using namespace std;

// Segment Tree Beats: supports range chmin (set all > x to x)
// and range sum query
// Based on the classic "Segment Tree Beats" by iwi
class SegTreeBeats {
private:
    struct Node {
        long long sum;
        int mx, smx, cnt; // max, second max, count of max
    };
    int n;
    vector<Node> tree;
    const int INF = 2e9;

    void build(const vector<int>& arr, int node, int l, int r) {
        if (l == r) {
            tree[node] = {arr[l], arr[l], -INF, 1};
            return;
        }
        int mid = (l + r) / 2;
        build(arr, 2 * node, l, mid);
        build(arr, 2 * node + 1, mid + 1, r);
        pull(node);
    }

```

```

}

void pull(int node) {
    Node& nd = tree[node];
    Node& left = tree[2 * node];
    Node& right = tree[2 * node + 1];
    nd.sum = left.sum + right.sum;
    if (left.mx == right.mx) {
        nd.mx = left.mx;
        nd.smx = max(left.smx, right.smx);
        nd.cnt = left.cnt + right.cnt;
    } else if (left.mx > right.mx) {
        nd.mx = left.mx;
        nd.smx = max(left.smx, right.mx);
        nd.cnt = left.cnt;
    } else {
        nd.mx = right.mx;
        nd.smx = max(left.mx, right.smx);
        nd.cnt = right.cnt;
    }
}

void apply(int node, int val) {
    if (tree[node].mx <= val) return;
    tree[node].sum -= (long long)(tree[node].mx - val) * tree[node].cnt;
    tree[node].mx = val;
}

void push(int node) {
    apply(2 * node, tree[node].mx);
    apply(2 * node + 1, tree[node].mx);
}

void rangeChmin(int node, int l, int r, int ql, int qr, int val) {
    if (ql > r || qr < l || tree[node].mx <= val) return;
    if (ql <= l && r <= qr && tree[node].smx < val) {
        apply(node, val);
        return;
    }
    push(node);
    int mid = (l + r) / 2;
    rangeChmin(2 * node, l, mid, ql, qr, val);
    rangeChmin(2 * node + 1, mid + 1, r, ql, qr, val);
    pull(node);
}

long long rangeSum(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return tree[node].sum;
    push(node);
    int mid = (l + r) / 2;
    return rangeSum(2 * node, l, mid, ql, qr) +
           rangeSum(2 * node + 1, mid + 1, r, ql, qr);
}

```

```

public:
    SegTreeBeats(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }

    void chmin(int l, int r, int val) {
        rangeChmin(1, 0, n - 1, l, r, val);
    }

    long long sum(int l, int r) {
        return rangeSum(1, 0, n - 1, l, r);
    }
};

int main() {
    vector<int> arr = {3, 1, 4, 1, 5, 9, 2, 6};
    SegTreeBeats st(arr);
    cout << "Sum [0,7]: " << st.sum(0, 7) << "\n"; // 31
    st.chmin(2, 5, 3); // set all > 3 in [2,5] to 3
    cout << "Sum [0,7]: " << st.sum(0, 7) << "\n"; // 3+1+3+1+3+3+2+6 = 22
    return 0;
}

```

Persistent Segment Tree

```

#include <bits/stdc++.h>
using namespace std;

// Persistent Segment Tree for Range Sum
// Each update creates a new version, preserving old ones
struct Node {
    int val;
    Node *left, *right;
    Node(int v = 0) : val(v), left(nullptr), right(nullptr) {}
};

class PersistentSegTree {
private:
    int n;
    vector<Node*> roots; // roots[version] = root node

    Node* build(const vector<int>& arr, int l, int r) {
        Node* node = new Node();
        if (l == r) {
            node->val = arr[l];
            return node;
        }
        int mid = (l + r) / 2;

```

```

        node->left = build(arr, l, mid);
        node->right = build(arr, mid + 1, r);
        node->val = node->left->val + node->right->val;
        return node;
    }

Node* update(Node* prev, int l, int r, int pos, int val) {
    Node* node = new Node();
    if (l == r) {
        node->val = val;
        return node;
    }
    int mid = (l + r) / 2;
    if (pos <= mid) {
        node->left = update(prev->left, l, mid, pos, val);
        node->right = prev->right;
    } else {
        node->left = prev->left;
        node->right = update(prev->right, mid + 1, r, pos, val);
    }
    node->val = node->left->val + node->right->val;
    return node;
}

int query(Node* node, int l, int r, int ql, int qr) {
    if (!node || ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return node->val;
    int mid = (l + r) / 2;
    return query(node->left, l, mid, ql, qr) +
           query(node->right, mid + 1, r, ql, qr);
}

public:
    PersistentSegTree(const vector<int>& arr) {
        n = arr.size();
        roots.push_back(build(arr, 0, n - 1));
    }

    void pointUpdate(int pos, int val) {
        roots.push_back(update(roots.back(), 0, n - 1, pos, val));
    }

    int rangeQuery(int version, int l, int r) {
        return query(roots[version], 0, n - 1, l, r);
    }

    int getVersionCount() { return roots.size(); }
};

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    PersistentSegTree pst(arr);

    cout << "V0 sum [1,3]: " << pst.rangeQuery(0, 1, 3) << "\n"; // 9
}

```

```

    pst.pointUpdate(2, 10); // version 1
    cout << "V1 sum [1,3]: " << pst.rangeQuery(1, 1, 3) << "\n"; // 2+10+4 = 16
    cout << "V0 sum [1,3]: " << pst.rangeQuery(0, 1, 3) << "\n"; // 9 (unchanged)
    return 0;
}

```

Dynamic Segment Tree (On-Demand Node Allocation)

```

#include <bits/stdc++.h>
using namespace std;

// Dynamic Segment Tree for range sum with large coordinate range
// Nodes are created only when needed
struct Node {
    long long val;
    Node *left, *right;
    Node() : val(0), left(nullptr), right(nullptr) {}
};

class DynamicSegTree {
private:
    long long n;
    Node* root;

    void update(Node* node, long long l, long long r, long long pos, long long val) {
        if (l == r) {
            node->val += val; // for increment
            return;
        }
        long long mid = (l + r) / 2;
        if (pos <= mid) {
            if (!node->left) node->left = new Node();
            update(node->left, l, mid, pos, val);
        } else {
            if (!node->right) node->right = new Node();
            update(node->right, mid + 1, r, pos, val);
        }
        long long leftVal = node->left ? node->left->val : 0;
        long long rightVal = node->right ? node->right->val : 0;
        node->val = leftVal + rightVal;
    }

    long long query(Node* node, long long l, long long r, long long ql, long long qr) {
        if (!node || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return node->val;
        long long mid = (l + r) / 2;
        return query(node->left, l, mid, ql, qr) +
               query(node->right, mid + 1, r, ql, qr);
    }
}

```

```

public:
    DynamicSegTree(long long size) : n(size) {
        root = new Node();
    }

    void pointAdd(long long pos, long long val) {
        update(root, 0, n - 1, pos, val);
    }

    long long rangeSum(long long l, long long r) {
        return query(root, 0, n - 1, l, r);
    }
};

int main() {
    // Array size up to 10^9, but only 5 updates
    DynamicSegTree dst(1e9);
    dst.pointAdd(100, 5);
    dst.pointAdd(1000000000 - 1, 10);
    dst.pointAdd(500000000, 3);
    cout << "Sum [0, 1e9-1]: " << dst.rangeSum(0, 1e9 - 1) << "\n"; // 18
    cout << "Sum [0, 100]: " << dst.rangeSum(0, 100) << "\n"; // 5
    return 0;
}

```

2D Segment Tree

```

#include <bits/stdc++.h>
using namespace std;

// 2D Segment Tree for range sum on a matrix
// Outer tree over rows, each node has an inner tree over columns
class SegTree2D {
private:
    int n, m;
    vector<vector<int>>> tree;

    void buildY(int nodeX, int lx, int rx, int nodeY, int ly, int ry,
                const vector<vector<int>>>& mat) {
        if (ly == ry) {
            if (lx == rx)
                tree[nodeX][nodeY] = mat[lx][ly];
            else
                tree[nodeX][nodeY] = tree[2 * nodeX][nodeY] + tree[2 * nodeX + 1][nodeY];
            return;
        }
        int midY = (ly + ry) / 2;
        buildY(nodeX, lx, rx, 2 * nodeY, ly, midY, mat);
        buildY(nodeX, lx, rx, 2 * nodeY + 1, midY + 1, ry, mat);
    }
};

```

```

    tree[nodeX][nodeY] = tree[nodeX][2 * nodeY] + tree[nodeX][2 * nodeY + 1];
}

void buildX(int nodeX, int lx, int rx, const vector<vector<int>>& mat) {
    if (lx == rx) {
        buildY(nodeX, lx, rx, 1, 0, m - 1, mat);
        return;
    }
    int midX = (lx + rx) / 2;
    buildX(2 * nodeX, lx, midX, mat);
    buildX(2 * nodeX + 1, midX + 1, rx, mat);
    buildY(nodeX, lx, rx, 1, 0, m - 1, mat);
}

void updateY(int nodeX, int lx, int rx, int nodeY, int ly, int ry,
             int y, int val) {
    if (ly == ry) {
        if (lx == rx)
            tree[nodeX][nodeY] = val;
        else
            tree[nodeX][nodeY] = tree[2 * nodeX][nodeY] + tree[2 * nodeX + 1]
[nodeY];
        return;
    }
    int midY = (ly + ry) / 2;
    if (y <= midY)
        updateY(nodeX, lx, rx, 2 * nodeY, ly, midY, y, val);
    else
        updateY(nodeX, lx, rx, 2 * nodeY + 1, midY + 1, ry, y, val);
    tree[nodeX][nodeY] = tree[nodeX][2 * nodeY] + tree[nodeX][2 * nodeY + 1];
}

void updateX(int nodeX, int lx, int rx, int x, int y, int val) {
    if (lx == rx) {
        updateY(nodeX, lx, rx, 1, 0, m - 1, y, val);
        return;
    }
    int midX = (lx + rx) / 2;
    if (x <= midX)
        updateX(2 * nodeX, lx, midX, x, y, val);
    else
        updateX(2 * nodeX + 1, midX + 1, rx, x, y, val);
    updateY(nodeX, lx, rx, 1, 0, m - 1, y, val);
}

int queryY(int nodeX, int nodeY, int ly, int ry, int qly, int qry) {
    if (qly > ry || qry < ly) return 0;
    if (qly <= ly && ry <= qry) return tree[nodeX][nodeY];
    int midY = (ly + ry) / 2;
    return queryY(nodeX, 2 * nodeY, ly, midY, qly, qry) +
        queryY(nodeX, 2 * nodeY + 1, midY + 1, ry, qly, qry);
}

int queryX(int nodeX, int lx, int rx, int qlx, int qrx, int qly, int qry) {

```

```

        if (qlx > rx || qrx < lx) return 0;
        if (qlx <= lx && rx <= qrx)
            return queryY(nodeX, 1, 0, m - 1, qly, qry);
        int midX = (lx + rx) / 2;
        return queryX(2 * nodeX, lx, midX, qlx, qrx, qly, qry) +
            queryX(2 * nodeX + 1, midX + 1, rx, qlx, qrx, qly, qry);
    }

public:
    SegTree2D(const vector<vector<int>>& mat) {
        n = mat.size();
        m = mat[0].size();
        tree.assign(4 * n, vector<int>(4 * m, 0));
        buildX(1, 0, n - 1, mat);
    }

    void pointUpdate(int x, int y, int val) {
        updateX(1, 0, n - 1, x, y, val);
    }

    int rangeQuery(int x1, int y1, int x2, int y2) {
        return queryX(1, 0, n - 1, x1, x2, y1, y2);
    }
};

int main() {
    vector<vector<int>> mat = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    SegTree2D st(mat);
    cout << "Sum submatrix [0,0] to [2,2]: " << st.rangeQuery(0, 0, 2, 2) << "\n";
    // 45
    st.pointUpdate(1, 1, 50); // set (1,1) from 5 to 50
    cout << "Sum submatrix [0,0] to [2,2]: " << st.rangeQuery(0, 0, 2, 2) << "\n";
    // 45 - 5 + 50 = 90
    return 0;
}

```

9. Python Implementation

Basic Segment Tree — Range Sum Query, Point Update

```

from typing import List

class SegmentTree:
    def __init__(self, arr: List[int]):
        self.n = len(arr)

```

```

        self.tree = [0] * (4 * self.n)
        self._build(arr, 1, 0, self.n - 1)

    def _build(self, arr: List[int], node: int, l: int, r: int):
        if l == r:
            self.tree[node] = arr[l]
            return
        mid = (l + r) // 2
        self._build(arr, 2 * node, l, mid)
        self._build(arr, 2 * node + 1, mid + 1, r)
        self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def _update(self, node: int, l: int, r: int, pos: int, val: int):
        if l == r:
            self.tree[node] = val
            return
        mid = (l + r) // 2
        if pos <= mid:
            self._update(2 * node, l, mid, pos, val)
        else:
            self._update(2 * node + 1, mid + 1, r, pos, val)
        self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def _query(self, node: int, l: int, r: int, ql: int, qr: int) -> int:
        if ql > r or qr < l:
            return 0
        if ql <= l and r <= qr:
            return self.tree[node]
        mid = (l + r) // 2
        left = self._query(2 * node, l, mid, ql, qr)
        right = self._query(2 * node + 1, mid + 1, r, ql, qr)
        return left + right

    def point_update(self, pos: int, val: int):
        self._update(1, 0, self.n - 1, pos, val)

    def range_query(self, l: int, r: int) -> int:
        return self._query(1, 0, self.n - 1, l, r)

# Example usage
arr = [1, 3, 5, 7, 9, 11]
st = SegmentTree(arr)
print(f"Sum [1,4]: {st.range_query(1, 4)}") # 24
st.point_update(2, 10)
print(f"After update, sum [1,4]: {st.range_query(1, 4)}") # 29

```

Segment Tree with Lazy Propagation — Range Update, Range Sum

```
from typing import List
```

```

class LazySegmentTree:
    def __init__(self, arr: List[int]):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(arr, 1, 0, self.n - 1)

    def _build(self, arr: List[int], node: int, l: int, r: int):
        if l == r:
            self.tree[node] = arr[l]
            return
        mid = (l + r) // 2
        self._build(arr, 2 * node, l, mid)
        self._build(arr, 2 * node + 1, mid + 1, r)
        self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def _apply(self, node: int, l: int, r: int, val: int):
        self.tree[node] += val * (r - l + 1)
        self.lazy[node] += val

    def _push(self, node: int, l: int, r: int):
        if self.lazy[node] != 0:
            mid = (l + r) // 2
            self._apply(2 * node, l, mid, self.lazy[node])
            self._apply(2 * node + 1, mid + 1, r, self.lazy[node])
            self.lazy[node] = 0

    def _range_update(self, node: int, l: int, r: int, ul: int, ur: int, val:
int):
        if ul > r or ur < l:
            return
        if ul <= l and r <= ur:
            self._apply(node, l, r, val)
            return
        self._push(node, l, r)
        mid = (l + r) // 2
        self._range_update(2 * node, l, mid, ul, ur, val)
        self._range_update(2 * node + 1, mid + 1, r, ul, ur, val)
        self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def _range_query(self, node: int, l: int, r: int, ql: int, qr: int) -> int:
        if ql > r or qr < l:
            return 0
        if ql <= l and r <= qr:
            return self.tree[node]
        self._push(node, l, r)
        mid = (l + r) // 2
        return (self._range_query(2 * node, l, mid, ql, qr) +
                self._range_query(2 * node + 1, mid + 1, r, ql, qr))

    def range_add(self, l: int, r: int, val: int):
        self._range_update(1, 0, self.n - 1, l, r, val)

```

```
def range_sum(self, l: int, r: int) -> int:
    return self._range_query(1, 0, self.n - 1, l, r)

# Example usage
arr = [1, 2, 3, 4, 5]
st = LazySegmentTree(arr)
print(f"Sum [0,4]: {st.range_sum(0, 4)}") # 15
st.range_add(1, 3, 10)
print(f"Sum [0,4]: {st.range_sum(0, 4)}") # 45
```

10. Code Explanation

Build Function

The build function recursively constructs the segment tree:

1. **Base case** ($l == r$): We've reached a leaf. Store the array element at that index.
2. **Recursive case**: Compute `mid`, build left child `[l, mid]` and right child `[mid+1, r]`.
3. **Merge**: After children are built, the parent's value is the combination of children's values.

Why $4 * n$? A segment tree on an array of size n has at most $4 * n$ nodes in the array representation. For a power of two, it's $2 * 2^{\lceil \log_2(n) \rceil} - 1$ which is $\leq 4n$. This is a safe upper bound.

Point Update Function

1. **Base case** ($l == r$): We've reached the target leaf. Update the value.
2. **Recursive case**: Decide whether to go left or right based on `pos` relative to `mid`.
3. **Recompute**: After updating the child, recompute the current node's value by merging children.

Range Query Function

1. **No overlap** ($ql > r \ || \ qr < l$): Return identity element (0 for sum, INF for min).
2. **Full overlap** ($ql \leq l \ \&\& \ r \leq qr$): Return the node's stored value.
3. **Partial overlap**: Recurse to both children, merge results.

Lazy Propagation (Range Update)

The key insight: when updating a range, we don't go all the way down to every leaf. Instead:

1. If the current segment is fully inside the update range, we update the node's value and mark it as **lazy** (pending update to propagate later).
2. The lazy value represents "this operation needs to be applied to all children of this node."
3. When we need to access children (during a query or further update), we **push** the lazy value down first.
4. This ensures $O(\log n)$ per operation instead of $O(n)$.

apply function: Updates the node's value and accumulates the lazy value.

push function: Propagates lazy value to children before recursing.

Segment Tree Beats

The Beats variant maintains three extra fields per node:

- **mx**: maximum value in this segment
- **smx**: second maximum value (strictly less than mx)
- **cnt**: count of elements equal to mx

When performing **chmin(x)** (set all values $> x$ to x):

- If **mx** $\leq x$: nothing to do
- If **smx** $< x < \mathbf{mx}$: we can directly update the node's max values
- Otherwise: we must push down and recurse

This works because applying **chmin** only affects the maximum values. If the second max is less than x , then all elements equal to $\mathbf{mx}$ are the only ones that change, and we can update in $O(1)$.

Persistent Segment Tree

Instead of modifying nodes in-place, each update creates new nodes along the path from root to leaf:

- Old nodes remain unchanged (they belong to previous versions).
- New nodes share unchanged children with old nodes.
- The root of the new version is a new node; old roots remain accessible.

Memory: $O((n + q) \log n)$ where q is the number of updates.

Dynamic Segment Tree

Instead of pre-allocating $4n$ nodes, we create nodes only when needed:

- Start with just the root node.
- When recursing into a child that doesn't exist, create it.
- Each node stores left/right child pointers.

Memory: $O(q \log n)$ where q is the number of operations, not $O(n)$.

2D Segment Tree

A tree of trees:

- The outer tree (X-tree) is built over rows.
- Each node of the outer tree has an inner tree (Y-tree) built over columns for that row range.
- Updates: walk the outer tree ($O(\log n)$), update the inner tree at each node ($O(\log m)$).
- Queries: walk the outer tree ($O(\log n)$), query the inner tree at each node ($O(\log m)$).

Complexity: $O(\log^2 n)$ per operation.

11. Complexity Analysis

Operation	Segment Tree	Lazy Segment Tree	Fenwick Tree (BIT)	Sparse Table
Build	$O(n)$	$O(n)$	$O(n \log n)$	$O(n \log n)$
Point Query	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
Point Update	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$
Range Query	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
Range Update	$O(n \log n)$	$O(\log n)$	$O(\log n)$ with BIT	N/A
Memory	$O(n)$	$O(n)$	$O(n)$	$O(n \log n)$

Detailed Breakdown

Variant	Build	Query	Update	Space
Basic Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
With Lazy Propagation	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Segment Tree Beats	$O(n)$	$O(\log n)$	$O(\log n)$ amortized	$O(n)$
Persistent Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O((n+q) \log n)$
Dynamic Segment Tree	$O(1)$	$O(\log R)$	$O(\log R)$	$O(q \log R)$
2D Segment Tree	$O(n \cdot m)$	$O(\log^2 n)$	$O(\log^2 n)$	$O(n \cdot m)$

Note: For Segment Tree Beats, the chmin operation has **amortized** $O(\log n)$ complexity. The amortized analysis relies on the potential method where each node's "max value" can only decrease a limited number of times.

12. Common Patterns

Pattern 1: Range Query with Point Update

How to identify: Multiple queries asking for sum/min/max in a range, with single element updates in between.

General approach: Build a standard segment tree with the appropriate merge function.

Example problems: Range Sum Query (LeetCode 307), Range Minimum Query (standard CP)

Pattern 2: Range Update with Range Query

How to identify: Queries ask for range aggregates, and updates add/subtract a value to a range.

General approach: Segment tree with lazy propagation.

Example problems: Array Range Operations (CSES), Range Updates and Sums (OJP)

Pattern 3: Multiple Queries with Different Operations

How to identify: Problem requires both range sum and range min (or other combinations) on the same array.

General approach: Store multiple values per node (e.g., sum, min, max) and update them all together.

Example problems: Range Queries with Multiple Aggregates

Pattern 4: Finding K-th Element in Range

How to identify: "Find the k-th smallest element in range [l, r]"

General approach: Use a persistent segment tree (also called Chairman Tree) or merge sort tree.

Example problems: K-th Number (SPOJ), Range K-th Smallest (LeetCode)

Pattern 5: Segment Tree Beats

How to identify: Operations like "set all values in [l, r] to $\min(x, a[i])$ " (range chmin) or "set all values to $\max(x, a[i])$ " (range chmax).

General approach: Segment Tree Beats maintains max, second max, count of max.

Example problems: Range Chmin, Range Add, Range Sum (CF 438D)

Pattern 6: Offline Queries with Coordinate Compression

How to identify: Large input values (up to 10^9) but few queries.

General approach: Compress values, then use a standard segment tree over compressed indices.

Example problems: Range Sum with Coordinate Compression

Pattern 7: 2D Range Queries

How to identify: Queries on submatrices of a grid.

General approach: 2D segment tree or Fenwick tree of Fenwick trees.

Example problems: Range Sum Query 2D (LeetCode 308), Submatrix Queries

13. Common Mistakes

Off-by-One Errors

- **Wrong:** `if (l == r)` should be the base case, but checking `if (l > r)` can cause infinite recursion.
- **Wrong:** Array indices from 0 to n-1, but passing n as the right bound.
- **Wrong:** In `query`, the no-overlap condition `ql > r || qr < l` — accidentally using `>=` or `<=`.

Wrong Initialization

- **Tree size:** `tree.resize(4 * n)` is safe. Using `2 * n` can cause segfault for non-power-of-two sizes.
- **Lazy array:** Must be initialized to 0 (or identity for the lazy operation).
- **Identity element:** Using 0 for min query returns wrong results. Use `INT_MAX` or `INF`.

Overflow

- **Sum tree:** For arrays with large values and many elements, sum can exceed 32-bit integer. Use `long long`.
- **Multiplication:** If storing product or doing range multiplication, overflow is even more likely.

Incorrect Lazy Propagation

- **Forgetting to push:** When recursing in a query, always push lazy before recursing.
- **Double applying:** Applying the lazy value to the node and then also applying it to children without pushing.
- **Wrong apply for sum:** `tree[node] += val * (r - l + 1)` — forgetting the segment length.
- **Wrong apply for min/max:** For range add with min query, you can't just add to the min value (it works, but be careful with lazy composition).

Merge Function Issues

- **Non-associative operations:** Segment tree only works for associative operations.
- **Wrong merge order:** For some operations (like string concatenation), order matters. Ensure consistent left-to-right.

Array Indexing

- **1-indexing vs 0-indexing:** Be consistent. If array is 0-indexed, root is at 1, and segments are `[0, n-1]`.
- **Forgetting to pass l and r parameters:** Each recursive call needs the current segment bounds.

Persistent Segment Tree

- **Memory leak:** Not freeing old nodes (acceptable in CP, but bad practice).
- **Sharing nodes:** When updating, the new node must share unchanged subtrees with the old node.
- **Version indexing:** Version 0 is the initial array. After k updates, available versions are 0..k.

2D Segment Tree

- **Memory:** $4 * n * 4 * m$ can be massive. Use dynamic allocation or coordinate compression.
- **Build complexity:** Building the inner tree for each outer node is $O(n m \log n \log m)$ if done naively.

14. Edge Cases

Edge Case	What to Check
Empty array	$n = 0$. Handle separately or ensure tree size is at least 1.
Single element	$n = 1$. Query and update should work on the only element.
All equal elements	Min, max, sum queries should all be straightforward.
Sorted ascending/descending	No special behavior, but tests boundary recursion.
Large values	Use <code>long long</code> for sum trees.

Edge Case	What to Check
Negative values	Min query handles them fine. Sum query with negative values works.
Query whole array	<code>query(0, n-1)</code> should return the root's value.
Query single element	<code>query(i, i)</code> should return <code>arr[i]</code> .
Update same element twice	Both updates should stick.
Update outside range	Should be handled gracefully (no-op).
Range update with $l > r$	Invalid input; handle gracefully.
Large n (10^6+)	Stack overflow if recursion is deep. Use iterative segment tree or increase stack size.
n not a power of 2	Standard segment tree handles this fine. Lazy propagation also works.
Multiple range updates overlapping	Lazy values should compose correctly.
Zero-length range	$l > r$ in query — return identity.

15. Variations

15.1 Iterative Segment Tree

What changes: Uses a bottom-up approach instead of recursion. The tree is stored in an array of size $2 * n$ (for n being a power of two).

When used: When recursion depth is a concern (large n), or for slightly faster performance.

Placement relevance: Moderate. Useful for CP for speed, but recursive is more intuitive for interviews.

15.2 Fenwick Tree (Binary Indexed Tree)

What changes: Simpler tree structure, only 1-indexed, supports prefix sums.

When used: When you only need prefix queries and point updates. Much simpler and faster.

Placement relevance: High. Very common in interviews.

15.3 Merge Sort Tree

What changes: Each node stores a sorted list of its segment elements. Query for "numbers $\leq k$ in range $[l, r]$ " becomes $O(\log^2 n)$.

When used: For range queries that need sorted order information (k -th smallest, count of elements $\leq x$).

Placement relevance: Moderate. Appears in harder problems.

15.4 Wavelet Tree

What changes: A tree that partitions the array by value ranges, not index ranges.

When used: For range counting, k-th smallest, range quantile queries.

Placement relevance: Low. Advanced CP topic.

15.5 Li Chao Segment Tree

What changes: Each node stores a line ($mx + c$). Used for dynamic convex hull / line container.

When used: For problems where you need to query max of lines at a given x , and lines are added dynamically.

Placement relevance: Low for placements, moderate for CP.

15.6 Segment Tree with Lazy on Min/Max

What changes: For range add + range min query with lazy, the apply function is `tree[node] += val`. The lazy value is propagated similarly.

When used: When you need range add and range min/max.

Placement relevance: High. Common in advanced problems.

15.7 Segment Tree Beats (Range Chmin/Chmax)

What changes: Stores max, second max, and count of max. Allows range chmin in amortized $O(\log n)$.

When used: When operations include "set all values $> x$ to x " (or $< x$ to x).

Placement relevance: Low for placements, high for CP.

16. Related Algorithms/Data Structures

Segment Tree vs Fenwick Tree (BIT)

Aspect	Segment Tree	Fenwick Tree
Operations	Range query, point/range update	Prefix query, point update
Range update	Yes (with lazy)	Yes (with range BIT)
Non-sum operations	Yes (min, max, gcd, etc.)	Only if invertible (like xor)
Code complexity	Higher	Lower
Memory	$4n$	n
Constant factor	Higher	Lower

Choose Fenwick Tree when: You only need sum/xor prefix queries and point updates.

Choose Segment Tree when: You need min/max/gcd, range updates, or custom operations.

Segment Tree vs Sparse Table

Aspect	Segment Tree	Sparse Table
Build	$O(n)$	$O(n \log n)$
Query	$O(\log n)$	$O(1)$
Update	$O(\log n)$	$O(n \log n)$
Memory	$O(n)$	$O(n \log n)$

Choose Sparse Table when: Array is static (no updates), and you need $O(1)$ range queries.

Choose Segment Tree when: There are updates.

Segment Tree vs SQRT Decomposition

Aspect	Segment Tree	SQRT Decomposition
Query	$O(\log n)$	$O(\sqrt{n})$
Update	$O(\log n)$	$O(1)$ or $O(\sqrt{n})$
Build	$O(n)$	$O(n)$
Code complexity	Medium	Low

Choose SQRT Decomposition when: Simplicity matters, n is small ($\leq 10^5$), and $O(\sqrt{n})$ per operation is acceptable.

Segment Tree vs Binary Indexed Tree for Range Updates

Two BITs can handle range updates and range sum queries. But for range min/max with range updates, you need segment tree with lazy propagation.

Segment Tree vs Order Statistic Tree

For problems like "find k -th smallest in current array", an order statistic tree (balanced BST with size tracking) or policy-based data structure (PBDS) is more appropriate.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Range Sum Query - Mutable	LeetCode 307	Basic segment tree, point update, range sum	Easy
Range Minimum Query	SPOJ	Range minimum query, static or with updates	Easy
Update the Array	CodeChef	Point update, range sum	Easy

Medium

Problem	Platform	Pattern	Difficulty
Range Sum Query 2D - Mutable	LeetCode 308	2D segment tree / BIT	Medium
Range Minimum Query with Point Updates	HackerEarth	RMQ with updates	Medium
XOR on Segment	Codeforces 242E	Range XOR update, range sum query	Medium
Array Range Operations	CSES	Range add, range sum with lazy	Medium

Hard

Problem	Platform	Pattern	Difficulty
K-th Number	SPOJ	Persistent segment tree, k-th smallest in range	Hard
Range Chmin, Range Add, Range Sum	Codeforces	Segment Tree Beats	Hard
Subarray Queries with Range Updates	Codeforces	Complex lazy segment tree	Hard
Dynamic Segment Tree with Large Coordinates	Codeforces 915E	Dynamic segment tree, large range	Hard
2D Segment Tree - Grid Queries	SPOJ	2D BIT / segment tree	Hard

18. Interview Explanation

"A segment tree is a binary tree data structure for answering range queries on an array while supporting updates. It stores aggregate information about segments of the array.

The root of the tree represents the entire array. Each node's children represent the left and right halves of its segment. Leaves represent individual elements. Every internal node stores the combined value of its children — for example, sum, min, or max.

To build the tree, we recursively split the array until we reach leaves, then combine results upward. This takes $O(n)$ time.

To query a range, we traverse the tree. At each node, if the node's segment is completely inside our query range, we return the stored value. If it's completely outside, we return an identity element. If it partially overlaps, we recurse into children and combine their results. This takes $O(\log n)$ time because any range can be decomposed into $O(\log n)$ disjoint nodes.

To update a value, we go down to the leaf, update it, and recompute all ancestors — also $O(\log n)$.

For range updates (adding a value to every element in a range), we use lazy propagation — we defer updates to children by marking nodes as 'lazy' and only propagate when necessary. This keeps range updates $O(\log n)$ as well.

The segment tree can be extended to support any associative operation, persistent versions, dynamic allocation for large ranges, and even 2D queries. The main trade-off is that it uses about $4n$ memory and has a higher constant factor than a Fenwick tree, which is simpler for prefix sums."

19. Revision Notes

Key Idea

- Binary tree over array indices. Each node covers a segment $[l, r]$.
- Root = entire array, leaves = single elements.
- Internal node stores merged result of children.

Build

```
build(node, l, r):
    if l == r: tree[node] = arr[l]
    else: mid = (l+r)/2, build children, tree[node] = merge(left, right)
```

- Time: $O(n)$, Space: $4n$ array

Point Update

```
update(node, l, r, pos, val):
    if l == r: tree[node] = val
    else: recurse to child containing pos, then tree[node] = merge(children)
```

- Time: $O(\log n)$

Range Query

```
query(node, l, r, ql, qr):
    if ql > r || qr < l: return identity
    if ql <= l && r <= qr: return tree[node]
    mid = (l+r)/2, return merge(query(left), query(right))
```

- Time: $O(\log n)$

Lazy Propagation (Range Update)

- $lazy[node]$ = pending value to add to segment

- `apply(node, l, r, val): tree[node] += val * (r-l+1), lazy[node] += val`
- `push(node, l, r):` propagate lazy to children before recursing
- Time: $O(\log n)$

Identity Values

Operation	Identity
Sum	0
Min	+INF
Max	-INF
XOR	0
GCD	0
Product	1

Common Traps

- Use `4 * n` for tree size, not `2 * n`
- Use `long long` for sum trees
- Push lazy **before** recursing in queries
- For min/max, identity must be INF/-INF, not 0
- Segment tree only works for **associative** operations

Complexity Cheat Sheet

Operation	Time
Build	$O(n)$
Point Query	$O(\log n)$
Point Update	$O(\log n)$
Range Query	$O(\log n)$
Range Update (lazy)	$O(\log n)$
Memory	$O(n)$

20. Final Cheat Sheet

SEGMENT TREE CHEAT SHEET

WHEN TO USE:

- Range queries with point updates (sum, min, max, gcd, xor)
- Range updates with range queries (lazy propagation)
- Custom associative merge operations

- 2D range queries
- Persistent data structures (historical queries)
- Large coordinate ranges (dynamic allocation)

MAIN OPERATIONS & COMPLEXITY:

Operation	Time	Notes
Build	$O(n)$	Recursive merge
Point Query	$O(\log n)$	Range query with $l=r$
Point Update	$O(\log n)$	Update leaf, recompute up
Range Query	$O(\log n)$	Decompose into $O(\log n)$ segs
Range Update	$O(\log n)$	With lazy propagation
Memory	$O(n)$	$4n$ array

KEY CODE IDEA:

```
// Build
void build(int node, int l, int r, vector<int>& arr) {
    if (l == r) { tree[node] = arr[l]; return; }
    int mid = (l + r) / 2;
    build(2*node, l, mid, arr);
    build(2*node+1, mid+1, r, arr);
    tree[node] = merge(tree[2*node], tree[2*node+1]);
}

// Query
int query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return IDENTITY;
    if (ql <= l && r <= qr) return tree[node];
    int mid = (l + r) / 2;
    return merge(query(2*node, l, mid, ql, qr),
                 query(2*node+1, mid+1, r, ql, qr));
}

// Point Update
void update(int node, int l, int r, int pos, int val) {
    if (l == r) { tree[node] = val; return; }
    int mid = (l + r) / 2;
    if (pos <= mid) update(2*node, l, mid, pos, val);
    else update(2*node+1, mid+1, r, pos, val);
    tree[node] = merge(tree[2*node], tree[2*node+1]);
}
```

IMPORTANT EDGE CASES:

- $n = 0$: Handle separately
- $n = 1$: Single element, query/update both work
- Large values: Use long long for sum
- Identity: 0 for sum, INF for min, -INF for max
- Lazy propagation: Must push before recursing
- Tree size: Always $4*n$ (not $2*n$)
- Offset: 0-indexed array, root at 1

WHEN NOT TO USE:

- Only prefix sums → Fenwick Tree (simpler, faster)
- Static array with no updates → Sparse Table ($O(1)$ query)
- Small n or simple problems → brute force
- Order statistics → PBDS / balanced BST